

BANKING TRANSACTION VISUALIZER USING DIRECTED GRAPHS

A DMS MINOR PROJECT REPORT SUBMITTED TO

THE NATIONAL INSTITUTE OF ENGINEERING, MYSURU
(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)



ESTD : 1946

in partial fulfillment for the award of degree of

Bachelor of Engineering

in

Computer Science and Engineering

Submitted By

Shreeja Hebbar 4NI24CS201

Under the guidance of

Mrs. Savitha Sridharamurthy
Assistant Professor

Department of CS&E
NIE, Mysuru



Department of Computer Science & Engineering
THE NATIONAL INSTITUTE OF ENGINEERING

(Autonomous Institution) Mysuru - 570 018

2025-26



THE NATIONAL INSTITUTE OF ENGINEERING

(An Autonomous Institute under Visvesvaraya
Technological University, Belagavi)



Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the mini project titled “**Banking Transaction Visualizer using Directed Graphs**” is the original work carried out by **Shreeja Hebbar (USN: 4NI24CS201)**, submitted in partial fulfillment of the requirements for the course Discrete Mathematical Structures. The project has been completed under the guidance of **Mrs. Savitha Sridharamurthy**, Assistant Professor, Department of Computer Science.

I sincerely thank Prof. Savitha Sridharamurthy for her invaluable guidance and continuous support throughout this project.

Guide Signature

Prof. Savitha Sridharamurthy

Assistant Professor

Department of Computer Science

ABSTRACT

This project, **Banking Transaction Visualizer using Directed Graphs**, addresses the real-world challenge of safely handling concurrent banking transactions, such as simultaneous ATM withdrawals, online deposits, balance inquiries, and fund transfers occurring on the same account at the same time. In modern banking systems, thousands of transactions are processed concurrently every second, and improper execution can lead to inconsistent account balances, data corruption, race conditions, or even financial loss. To solve this problem, the project applies fundamental concepts from **Discrete Mathematical Structures (DMS)** including **directed graphs, binary relations, conflict serializability, Depth-First Search (DFS), graph traversal, and cycle detection algorithms**.

The system simulates multiple banking transactions executing concurrently and analyzes the conflicts that arise when two or more transactions attempt to access the same data item simultaneously. By examining read-write, write-read, and write-write conflicts, the application automatically constructs a **precedence graph (serialization graph)** where each node represents a transaction and each directed edge represents a dependency or conflict relation between transactions. The graph visually models the order in which transactions must execute to maintain database consistency.

Using **DFS-based cycle detection**, the system determines whether the generated transaction schedule is **serializable** (safe for execution) or **non-serializable** (unsafe). If the precedence graph contains no cycles, the schedule is considered conflict-serializable and equivalent to a safe sequential execution. However, if a cycle is detected, the system identifies the conflicting transactions and prevents inconsistent execution by triggering rollback and recovery mechanisms. This demonstrates how graph theory and mathematical reasoning directly support concurrency control techniques used in real-world database management systems and core banking software.

The project successfully demonstrates that concepts taught in Discrete Mathematics are not merely theoretical topics but form the core foundation of practical technologies such as database concurrency control, operating systems, transaction management, deadlock detection, and modern banking infrastructures. By integrating mathematical theory with software engineering and visualization, the project provides both an educational learning platform and a practical simulation of real-world transaction processing systems.

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Abstract	III
Table of Contents	IV
Introduction	1
DMS Concept Used	2
Methodology	2
Implementation	3
Sample Output	4
Result	5
Complexity Analysis	6
Screenshots	7
Conclusion	9
References	9
Appendix	10

INTRODUCTION

OBJECTIVES

- Simulate concurrent banking transactions like deposit, withdrawal, and transfer.
- Detect transaction conflicts using binary relations.
- Generate a precedence graph for conflicting transactions.
- Use DFS to detect cycles and identify safe or unsafe schedules.
- Implement rollback recovery for unsafe transactions.
- Visualize graphs and transaction execution for better understanding.
- Demonstrate the application of DMS concepts in banking and database systems.
- Improve understanding of concurrency control and serializability concepts.
- Showcase the role of graph theory and algorithms in real-world financial systems.

PROBLEM STATEMENT

Modern banking systems handle millions of transactions simultaneously through ATMs, mobile banking, and online payment systems. When multiple users access the same account concurrently, incorrect execution order may lead to data inconsistency and financial loss. For example, two ATMs processing withdrawals at the same time may both read the same account balance before either update occurs, causing both transactions to succeed incorrectly. This creates the **Lost Update Problem**, where the bank dispenses more money than available in the account.

This project addresses such concurrency issues by modelling transactions mathematically using concepts from Discrete Mathematical Structures (DMS). Conflicts between transactions are represented using binary relations, while transaction dependencies are modelled through directed precedence graphs. Depth-First Search (DFS) is applied to detect cycles in the graph and determine whether the schedule is conflict-serializable and therefore safe. If a cycle is detected, rollback recovery restores account consistency.

The project demonstrates how graph theory, binary relations, and algorithms can be applied to real-world banking and database systems to ensure correctness, reliability, and safe concurrent transaction processing.

DMS CONCEPTS USED

This project applies several concepts of Discrete Mathematical Structures (DMS) to solve concurrency problems in banking systems. Directed graphs are used to represent transaction dependencies, where each transaction acts as a node and conflicts between transactions create directed edges in the precedence graph. Binary relations are applied to identify conflicts when two different transactions access the same account and at least one operation is a WRITE operation.

Depth-First Search (DFS) is used to traverse the precedence graph and detect cycles efficiently. If a cycle is present, the graph is not a Directed Acyclic Graph (DAG), indicating that the schedule is unsafe and non-serializable. If the graph is acyclic, the schedule is considered safe for execution. The project also uses graph traversal and mapping functions to convert detected conflicts into graph edges and analyze transaction ordering constraints.

The project demonstrates the practical use of graph theory, algorithms, and mathematical relations in real-world database systems. These concepts are widely used in banking servers, distributed systems, and transaction processing applications to maintain consistency and reliability during concurrent execution.

METHODOLOGY

The user enters transaction schedules through the GUI in the format: Transaction ID, operation type, account name, and amount. The system then checks all transaction pairs for conflicts and constructs a precedence graph representing transaction dependencies.

DFS cycle detection is applied to the graph using visited and recursion stack methods. If a cycle is detected, the schedule is marked unsafe and rollback recovery restores account balances to maintain consistency. If no cycle is found, the transactions are committed successfully.

Finally, the precedence graph is visualized using NetworkX and Matplotlib. Safe schedules are displayed in green, while unsafe schedules are shown in red. This helps users clearly understand transaction conflicts, graph traversal, and concurrency control concepts.

IMPLEMENTATION

Input Data: Sample Accounts

Account ID	Account Holder	Initial Balance
A	Alice	₹10,000
B	Bob	₹8,000
C	Carol	₹6,000
X	Xavier	₹5,000

Modules

Module / File	Responsibility
accounts.py	Account class — deposit, withdraw, balance checkpoint, rollback
transactions.py	Transaction class; schedule list builder from raw input
conflict_detector.py	Pairwise $O(n^2)$ conflict detection; returns list of conflict tuples
graph_generator.py	Builds NetworkX DiGraph; renders graph with Matplotlib
cycle_detection.py	Custom recursive DFS with recursion stack; cycle path extractor
rollback.py	Saves balance checkpoint; restores all accounts on rollback
scheduler.py	Executes schedule step-by-step; generates execution log
main.py	Integrates backend logic with the frontend interface and controls execution flow.

Tech Stack

- **Frontend:** HTML, CSS, Javascript — 4-tab GUI with schedule editor, execution log, graph display, quiz mode.
- **Backend:** Python 3.8+ — all logic modules (conflict detection, DFS, rollback, scheduling).
- **Tools:** NetworkX (graph data structure and layout), Matplotlib (graph rendering and embedding in Tkinter via FigureCanvasTkAgg).

SAMPLE OUTPUT

Scenario A — Safe Schedule

Schedule: T1 reads A, T1 writes A (-25,00), T2 reads B, T2 writes B (+3,000). T1 and T2 access different accounts. No conflict pairs exist. Precedence graph has 2 isolated vertices with no edges.

```
=====
[T1] READ  A (Alice): ₹10,000
[T1] WRITE A (Alice): -₹2,000 → ₹8,000
[T2] READ  B (Bob):   ₹8,000
[T2] WRITE B (Bob):  +₹3,000 → ₹11,000
-----
CONFLICTS DETECTED: 0
NO CYCLE FOUND
SCHEDULE IS SAFE ✓
=====
```

Scenario B — Unsafe Schedule (Write-Write Cycle)

Schedule: T1 reads A, T2 writes A, T1 writes A, T2 reads A. Two conflicts detected: T1→T2 (RW) and T2→T1 (WW). Cycle T1→T2→T1 detected. Rollback triggered.

```
=====
[T1] READ  A (Alice): ₹10,000
[T2] WRITE A (Alice): +₹1,000 → ₹11,000
[T1] WRITE A (Alice): -₹500 → ₹10,500
[T2] READ  A (Alice): ₹10,500
-----
CONFLICTS DETECTED: 2
  T1 ↔ T2 on A [READ vs WRITE]
  T2 ↔ T1 on A [WRITE vs WRITE]
CYCLE DETECTED: T1 → T2 → T1
SCHEDULE IS UNSAFE ✗
[ROLLBACK] Alice (A): ₹10,500 → ₹10,000
=====
```

Scenario C — Three-Transaction Linear Chain (Safe)

Schedule: T1 writes A, T2 reads A, T2 writes B, T3 reads B. Conflicts: T1→T2 (WR on A), T2→T3 (WR on B). Linear chain T1→T2→T3 — no cycle — SAFE.

Precedence Graph: $T1 \rightarrow T2 \rightarrow T3$
 Equivalent serial order: T1, T2, T3
 SCHEDULE IS SAFE ✓

RESULTS

The Banking Transaction Visualizer was tested against all four built-in preset scenarios and multiple custom schedules. The following results were consistently obtained:

Test Scenario	Conflicts	Graph	Cycle	Verdict
Safe: T1 on A, T2 on B	0	Isolated vertices	No	SAFE ✓
Unsafe: Write-Write on A	2	$T1 \rightarrow T2, T2 \rightarrow T1$	Yes	UNSAFE + Rollback
Classic Cycle $T1 \leftrightarrow T2$	2	$T1 \rightarrow T2, T2 \rightarrow T1$	Yes	UNSAFE + Rollback
3-Transaction Chain	2	$T1 \rightarrow T2 \rightarrow T3$	No	SAFE ✓

The GUI correctly:

- Detected all conflicts for every test scenario with no false positives or false negatives.
- Built the precedence graph automatically and rendered it with correct directed edges and labels.
- Applied DFS cycle detection accurately: graphs with cycles were always flagged UNSAFE; acyclic graphs were always flagged SAFE.
- Restored account balances to the pre-schedule checkpoint on every triggered rollback.
- Displayed quiz mode answers correctly with revealed precedence graphs and explanations.
- Logged all transaction execution steps, conflict detections, cycle paths, and rollback operations in real time, helping users clearly trace the behavior of concurrent schedules step-by-step.

COMPLEXITY ANALYSIS

Time Complexity

Operation	Algorithm	Time Complexity	Notes
Conflict Detection	Pairwise comparison	$O(n^2)$	n = number of schedule steps; all pairs checked
Graph Construction	Edge insertion into DiGraph	$O(E)$	E = number of conflict pairs
DFS Cycle Detection	Recursive DFS with rec. stack	$O(V + E)$	V = transactions, E = conflict edges
Cycle Path Extraction	NetworkX find_cycle	$O(V + E)$	Traversal to trace back edge path
Rollback	Dictionary lookup + assignment	$O(k)$	k = number of accounts to restore
TOTAL (Full Pipeline)	—	$O(n^2 + V + E)$	Dominated by conflict detection step

Space Complexity

Component	Space Used	Notes
Schedule storage	$O(n)$	List of n operation dicts
Conflict list	$O(c)$	$c \leq n(n-1)/2$ conflicts
Precedence graph	$O(V + E)$	NetworkX adjacency structure
DFS visited + rec_stack	$O(V)$	At most $ T $ nodes on stack at once
Rollback checkpoint	$O(k)$	One balance value per account
TOTAL	$O(n^2 + V + E)$	Conflict list storage dominates for dense schedules

SCREENSHOTS

CORE BANKING OPERATIONS CONSOLE

Banking Transaction Visualizer

Ready

Dashboard

Deposit / Withdraw

Precedence Graphs

Quiz Mode

LEDGER VOLUME

₹12,700

ACTIVE TRANSACTIONS

7

CONFLICT EDGES

20

RISK STATE

Unsafe

ACCOUNT A

SAVINGS

Alice Sharma

A/C **** 4821 | RBI insured ledger

₹10,000

ACCOUNT B

CURRENT

Bob Mehta

A/C **** 7394 | RBI insured ledger

₹8,000

ACCOUNT C

SALARY

Carol Rao

A/C **** 1180 | RBI insured ledger

₹6,000

ACCOUNT D

PREMIUM

Dev Patel

A/C **** 9042 | RBI insured ledger

₹15,000

ACCOUNT E

BUSINESS

Enterprise Escrow

A/C **** 6675 | RBI insured ledger

₹22,000

ACCOUNT X

NRI

Xavier Khan

A/C **** 5509 | RBI insured ledger

₹5,000

ACCOUNT COUNTER

Deposit / Withdraw

Process branch deposits, ATM withdrawals, and fund transfers with live balance updates.

RISK ENGINE

Precedence Graphs

Run batches with more than five transactions, track conflicts, and detect unsafe cycles.

LEARNING MODE

Quiz Mode

Predict whether a banking schedule is safe before revealing the graph result.

ACCOUNT OPERATIONS DESK

Deposit / Withdraw Accounts

Withdrawal Posted

Dashboard

Deposit / Withdraw

Precedence Graphs

Quiz Mode

ACCOUNT A

SAVINGS

Alice Sharma

A/C **** 4821 | RBI insured ledger

₹10,900

ACCOUNT B

CURRENT

Bob Mehta

A/C **** 7394 | RBI insured ledger

₹8,000

ACCOUNT C

SALARY

Carol Rao

A/C **** 1180 | RBI insured ledger

₹6,000

ACCOUNT D

PREMIUM

Dev Patel

A/C **** 9042 | RBI insured ledger

₹15,000

ACCOUNT E

BUSINESS

Enterprise Escrow

A/C **** 6675 | RBI insured ledger

₹22,000

ACCOUNT X

NRI

Xavier Khan

A/C **** 5509 | RBI insured ledger

₹5,000

TELLER WINDOW

Deposit / Withdraw

ACCOUNT

A - Alice Sharma

AMOUNT

100

Deposit

Withdraw

Reset

INTERNAL TRANSFER

Fund Transfer

FROM

A - Alice Sharma

TO

B - Bob Mehta

AMOUNT

1500

Transfer Funds

AUDIT TRAIL

Transaction History

12:29:50 PM

WITHDRAW A ₹100 | Balance ₹10,900

12:29:43 PM

DEPOSIT A ₹1,000 | Balance ₹11,000

LEDGER QUEUE

Transaction Batch

Core Banking Rush

T1 read A 0
T2 transfer A B 2200
T3 read C 0
T4 withdraw B 700
T5 deposit A 900
T6 transfer C A 1200
T7 transfer D E 5000
T3 withdraw C 800
T4 read A 0
T6 deposit E 600
T5 transfer E B 1300
T7 read B 0

Run Animation
Reset
Rollback

LIVE CORE

Processing Stream

12 / 12

EDGE 07 T7 TRANSFER | D->E ₹5,000
IMPS

EDGE 08 T3 WITHDRAW | ATM
C -₹800

EDGE 09 T4 READ | Balance API
A 0

EDGE 10 T6 DEPOSIT | Branch
E +₹600

EDGE 11 T5 TRANSFER | E->B ₹1,300
IMPS

EDGE 12 T7 READ | Balance API
B 0

20 conflict edge(s): T1->T2 on A, T1->T5 on A, T1->T6 on A, T2->T4 on B, T2->T5 on A, T2->T6 on A, T2->T4 on A, T2->T5 on B, T2->T7 on B, T3->T6 on C, T4->T5 on B, T4->T7 on B, T5->T6 on A, T5->T4 on A, T6->T3 on C, T6->T4 on A, T7->T6 on E, T7->T5 on E, T6->T5 on E, T5->T7 on B

RISK GRAPH

Precedence Network

Unsafe

Cycle: T6 -> T3 -> T6
20 conflict edge(s)

AUDIT TRAIL

Ledger Journal

T6 -> T4 on A [TRANSFER(A) vs READ(A)]
T7 -> T6 on E [TRANSFER(E) vs DEPOSIT(E)]
T7 -> T5 on E [TRANSFER(E) vs TRANSFER(E)]
T6 -> T5 on E [DEPOSIT(E) vs TRANSFER(E)]
T5 -> T7 on B [TRANSFER(B) vs READ(B)]

CYCLE DETECTED: T6 -> T3 -> T6
ROLLBACK RECOMMENDED

SERIALIZABLE SCHEDULE LAB

Quiz Mode

Safe

Dashboard
Deposit / Withdraw
Precedence Graphs
Quiz Mode

PREDICTION

Safe or Unsafe?

3 / 6

01 T1 TRANSFER A->B ₹500
02 T2 READ B 0
03 T2 WITHDRAW B -₹300

Safe
Unsafe
Next

Correct. T1 must precede T2 on account B, but there is no edge back from T2 to T1.

REVEAL

Quiz Graph

Safe

No cycle found
2 conflict edge(s)

CONCLUSION

The Banking Transaction Visualizer using Directed Graphs successfully implemented safe transaction scheduling and concurrency analysis features. The system supports deposit, withdrawal, and fund transfer operations with concurrent transaction simulation, automatic conflict detection, precedence graph generation, DFS-based cycle detection, rollback recovery, and graphical visualization. Testing with different transaction schedules showed that the system accurately identified safe and unsafe schedules and maintained account consistency during rollback operations.

Discrete Mathematical Structures (DMS) provided the theoretical foundation for the project. Binary relations were used for conflict detection, directed graphs represented transaction dependencies, and DFS efficiently detected cycles in the precedence graph. The Conflict Serializability Theorem helped verify whether a transaction schedule was safe by checking if the graph was acyclic. This demonstrated the practical application of graph theory and discrete mathematics in banking and database concurrency control systems.

Future improvements may include Python threading for real parallel execution, SQLite integration for persistent storage, and support for advanced locking mechanisms and distributed banking models. Overall, the project effectively connects DMS concepts with practical software engineering and provides a strong educational model for transaction management and concurrency control.

REFERENCES

1. Rosen, K. H. – *Discrete Mathematics and Its Applications*, McGraw-Hill, 2019.
2. Ramakrishnan, R. & Gehrke, J. – *Database Management Systems*, McGraw-Hill, 2003.
3. Silberschatz, A., Korth, H. F. & Sudarshan, S. – *Database System Concepts*, McGraw-Hill, 2020.
4. Cormen, T. H. et al. – *Introduction to Algorithms*, MIT Press, 2009.
5. NetworkX Documentation – <https://networkx.org>
6. Matplotlib Documentation – <https://matplotlib.org>
7. Python Tkinter Documentation – <https://docs.python.org/3/library/tkinter.html>
8. Elmasri, R. & Navathe, S. B. – *Fundamentals of Database Systems*, Pearson Education, 2016.

APPENDIX

The following presents the key components of the project source code in condensed form. The full project consists of 8 Python files totalling approximately 650 lines of code.

accounts.py — Account Class

```
class Account:
    def __init__(self, account_id, name, balance):
        self.account_id = account_id
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0: raise ValueError('Must be positive')
        self.balance += amount

    def withdraw(self, amount):
        if self.balance < amount:
            raise ValueError('Insufficient balance')
        self.balance -= amount

    def save_checkpoint(self): self._cp = self.balance
    def rollback(self): self.balance = self._cp
```

conflict_detector.py — Conflict Detection

```
def detect_conflicts(schedule):
    conflicts = []
    for i in range(len(schedule)):
        for j in range(i+1, len(schedule)):
            s1, s2 = schedule[i], schedule[j]
            if s1['tid'] == s2['tid']: continue
            if s1['account'] != s2['account']: continue
            if 'write' in [s1['op'], s2['op']]:
                conflicts.append((s1['tid'], s2['tid'],
                                s1['account']))
    return conflicts
```

cycle_detection.py — DFS Cycle Detection

```
def dfs(node, visited, rec_stack, graph):
```

```
visited.add(node)
rec_stack.add(node)
for neighbor in graph.get(node, []):
    if neighbor not in visited:
        if dfs(neighbor, visited, rec_stack, graph):
            return True
    elif neighbor in rec_stack:
        return True
rec_stack.remove(node)
return False

def has_cycle(G):
    graph = {n: list(G.successors(n)) for n in G.nodes}
    visited, rec_stack = set(), set()
    for node in graph:
        if node not in visited:
            if dfs(node, visited, rec_stack, graph):
                return True
    return False
```

Project Run Instructions

Browser frontend

Open this file in your browser: frontend/index.html

The browser version uses HTML, CSS, and JavaScript with separate dashboard pages:

- `frontend/index.html` - overview dashboard
- `frontend/accounts.html` - deposit, withdraw, and transfer accounts
- `frontend/graph.html` - precedence graphs, live tracking, conflicts, and rollback
- `frontend/quiz.html` - safe/unsafe schedule quiz mode